

数据访问 ORM

目录

一、 ORM 概述	3
1-1.什么是 ORM	3
1-2.为什么需要 ORM	3
二、 常用数据接口详解	3
2-1.数据库操作服务帮助类 DBServiceHelper	3
2-2.查询服务帮助类 QueryServiceHelper	7
2-3.实体操作服务帮助类 OperationServiceHelper	13
2-4.业务数据服务帮助类 BusinessDataServiceHelper	16
2-5.实体保存服务帮助类 SaveServiceHelper	18
2-6.删除实体操作服务类 DeleteServiceHelper	19
三、 操作实践	20

一、ORM 概述

1-1.什么是 ORM

对象关系映射(Object Relation Mapping)的简称,把实体模型映射到数据库中,提供对象的操作,在数据库中则体现为 CRUD。

开源 ORM 框架: Hibernate、Mybatis、JPA。

1-2.为什么需要 ORM

多数据库访问的需要: 应用主库、基础资料、工作流、日志、权限

自动处理繁琐的关联操作: 分录、子分录、多语言表、扩展表 (动态)、多选
基础资料

基于表单实体元数据,更符合业务逻辑

集成计算框架接口,可便捷地使用复杂的计算、大数据量的访问

二、常用数据接口详解

2-1.数据库操作服务帮助类 DBServiceHelper

kd.bos.servicehelper.DBServiceHelper 为数据库操作服务帮助类,起到对数据库操作上层进行封装,简化数据库操作的作用。

举例,生成实体主键:

```
String id = DBServiceHelper.genStringId();
```

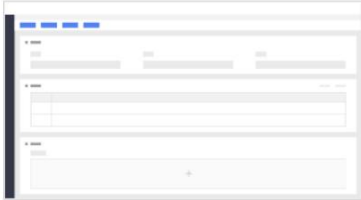
API 概览:

方法	说明
<code>static long genGlobalLongId()</code>	生成全局唯一的长整形主键
<code>static long[] genLongIds(String tableName, int count)</code>	批量生成长整形主键
<code>static String genStringId()</code>	生成字符型主键
<code>static String[] genStringIds(int count)</code>	批量生成字符型主键
<code>static String[] genStringIds(String tableName, int count)</code>	批量生成字符型主键

创建一张单据

页面创建向导-PC端

×



单据模板

编码*

qyrv_orctest

名称*

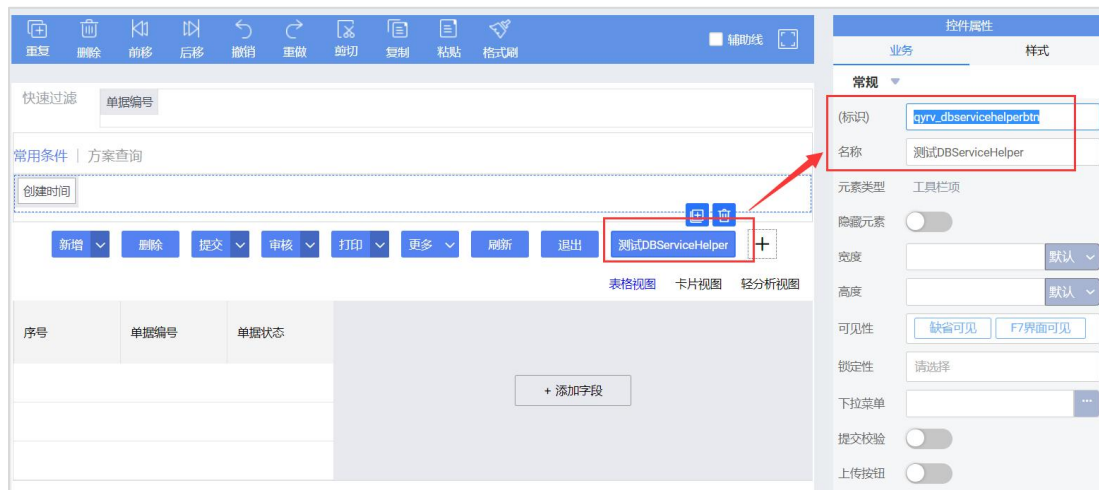
测试ORM

🔄

上一步

完成创建

在列表上增加一个按钮“测试 DBServiceHelper”，标识为“qyrv_dbsevicehelperbtn”



编写插件代码

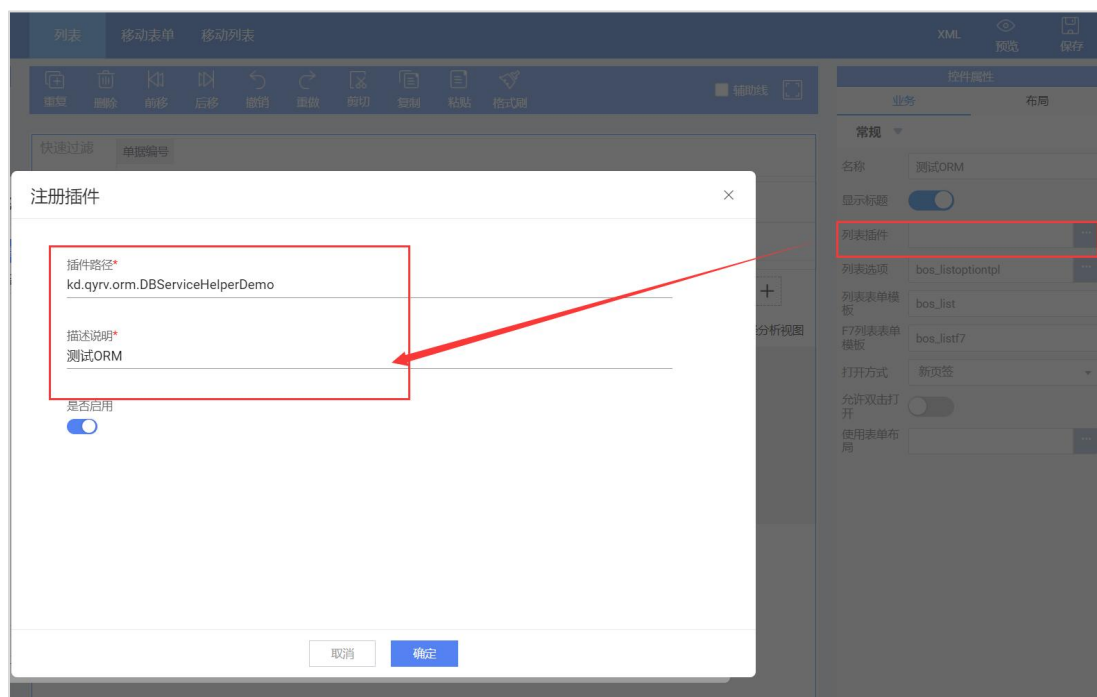
```
public class DBServiceHelperDemo extends AbstractListPlugin {  
    @Override  
    public void registerListener(EventObject e) {  
        super.registerListener(e);  
        this.addClickListeners(new String[] { "qyrv_dbservicehelperbtn" });  
    }  
    @Override  
    public void itemClick(ItemClickEvent evt) {  
        super.itemClick(evt);  
        // 获取事件源  
        String key = evt.getItemKey();  
        // 生成实体主键  
        if ("qyrv_dbservicehelperbtn".equals(key)) {  
            // 生成全局唯一的长整形主键  
            long id1 = DBServiceHelper.genGlobalLongId();  
            System.out.println("生成全局唯一的长整形主键:" + id1);  
        }  
    }  
}
```

```

// 批量生成长整型主键
System.out.println("批量生成长整型主键:");
// tk_qyrv_triptype 为表名
long[] ids2 = DBServiceHelper.genLongIds("tk_qyrv_triptype", 5);
for (long l : ids2) {
    System.out.println(l);
}
// 生成字符型主键
String id2 = DBServiceHelper.genStringId();
System.out.println("生成字符型主键: " + id2);

System.out.println("批量生成字符型主键");
//批量生成字符型主键
String[] ids3 = DBServiceHelper.genStringIds(3);
for (String s : ids3) {
    System.out.println(s);
}
System.out.println("批量生成字符型主键 2");
//批量生成字符型主键
String[] ids4 = DBServiceHelper.genStringIds("tk_qyrv_triptype", 3);
for (String s : ids4) {
    System.out.println(s);
}
}
}
}

```



演示效果

生成全局唯一的长整形主键:1707468356140924928

批量生成长整型主键:

1707468356140936192

1707468356140936193

1707468356140936194

1707468356140936195

1707468356140936196

生成字符型主键: 3BQA3P9R0PF4

批量生成字符型主键

3BQA3P9R0WV1

3BQA3P9R0WV2

3BQA3P9R0WV3|

批量生成字符型主键2

3BQA3P9R0PF5

3BQA3P9R0PF6

3BQA3P9R0PF7

2-2.查询服务帮助类 QueryServiceHelper

kd.bos.servicehelper.QueryServiceHelper 查询服务帮助类封装了一些查询方法。

举例，查询某条记录是否存在：

```
boolean exist = QueryServiceHelper.exists(entityName,pkId);
```

限定符和类型	方法和说明
public static boolean	exists (String entityName, Object pkId) 判断单据主键是否存在
public static boolean	exists (String entityName, QFilter[] qFilters) 判断符合条件的单据是否存在
public static java.util.List<kd.bos.servicehelper.BillNoDto>	getAllBillNo (String searchKey, int top) 获取所有的单据编号字段标识
public static DynamicObjectCollection	query (String entityName, String selectFields, QFilter[] filters) 查询单据，返回拉平的数据包
public static DynamicObjectCollection	query (String entityName, String selectFields, QFilter[] filters, String orderBys) 查询单据（排序），返回拉平的数据包
public static DynamicObjectCollection	query (String entityName, String selectFields, QFilter[] filters, String orderBys, int top) 查询单据（排序，前top条），返回拉平的数据包
public static DynamicObjectCollection	query (String algoKey, String entityName, String selectFields, QFilter[] filters, String orderBys) 查询单据（排序），返回拉平的数据包
public static DynamicObjectCollection	query (String algoKey, String entityName, String selectFields, QFilter[] filters, String orderBys, int top) 查询单据（排序，前top条），返回拉平的数据包
public static DynamicObjectCollection	queryByDate (String entityName, String selectFields, QFilter[] filters, java.util.Date date) 查询单据，返回拉平的数据包（支持获取名称版本化数据）
public static DataSet	queryDataSet (String algoKey, String entityName, String selectFields, QFilter[] filters, String orderBys) 查询单据，返回拉平的数据流
public static DataSet	queryDataSet (String algoKey, String entityName, String selectFields, QFilter[] filters, String orderBys, int top) 查询单据（排序，取top条），返回拉平的数据流
public static DynamicObject	queryOne (String entityName, String selectFields, QFilter[] filters) 查询一条数据（第一条）
public static java.util.List<Object>	queryPrimaryKeys (String entityName, QFilter[] filters, String orderBys, int top) 查询主键（排序，取top条）
public static java.util.List<Object>	queryPrimaryKeys (String algoKey, String entityName, QFilter[] filters, String orderBys, int top) 查询主键（排序，取top条）
public static java.util.List<Object>	queryPrimaryKeys (String algoKey, IDataEntityType entityType, QFilter[] filters, String orderBys, int top) 查询主键（排序，取top条）

在上述案例的基础上，在列表工具栏中分别增加如下按钮用于测试 QueryServiceHelper

不同方法的调用

qyrv_exists：测试 exists 方法

qyrv_getallbillno：获取前几条单据编号字段标识

qyrv_query：测试查询方法

qyrv_queryorder：查询排序

qyrv_queryordertop：查询排序前几条

qyrv_queryalgo：查询 algo

qyrv_querydataset：查询返回 DataSet

qyrv_queryone: 查询单条数据

qyrv_queryprimarykeys: 查询主键 Keys



```
@Override
public void itemClick(ItemClickEvent evt) {
    super.itemClick(evt);
    // 获取事件源
    String key = evt.getItemKey();
    switch (key) {
        case "qyrv_exists": // 查询某条记录是否存在
            // 根据pid判断数据是否存在
            // boolean flag = QueryServiceHelper.exists("qyrv_triptype",
            // "1402890214707298304");
            // System.out.println(flag?"存在":"不存在");
            // 根据过滤条件判断数据是否存在
            QFilter[] qFilters = new QFilter[] { new QFilter("status", QCP.equals,
"A") };

            boolean flag = QueryServiceHelper.exists("qyrv_triptype", qFilters);
            System.out.println(flag ? "存在" : "不存在");
            break;
    }
}
```

```
        case "qyrv_getallbillno": // 获取前 top 条单据编号字段标识
            List<BillNoDto> billnos =
QueryServiceHelper.getAllBillNo("qyrv_travreqbill", 5);
            for (BillNoDto dto : billnos) {
                System.out.println(dto.getBillNo());
            }
            break;
        case "qyrv_query": // 查询
            DynamicObjectCollection collections =
QueryServiceHelper.query("qyrv_triptype", "number,name,status",
                new QFilter[] { new QFilter("status", QCP.equals, "A") });
            Iterator<DynamicObject> it = collections.iterator();
            while (it.hasNext()) {
                DynamicObject obj = it.next();
                System.out.println(obj.get(0) + ":" + obj.get(1) + ":" + obj.get(2));
            }
            break;
        case "qyrv_queryorder": // 查询排序
            DynamicObjectCollection collections2 =
QueryServiceHelper.query("qyrv_triptype", "number,name,status",
                new QFilter[] { new QFilter("status", QCP.equals, "A") }, "number
desc");

            Iterator<DynamicObject> it2 = collections2.iterator();
            while (it2.hasNext()) {
                DynamicObject obj = it2.next();
                System.out.println(obj.get(0) + ":" + obj.get(1) + ":" + obj.get(2));
            }
            break;
```

```

        case "qyrv_queryordertop": // 查询排序前几条
            DynamicObjectCollection collections3 =
                QueryServiceHelper.query("qyrv_triptype", "number,name,status",
                    new QFilter[] { new QFilter("status", QCP.equals, "A") }, "number
                    desc", 2);

            Iterator<DynamicObject> it3 = collections3.iterator();
            while (it3.hasNext()) {
                DynamicObject obj = it3.next();
                System.out.println(obj.get(0) + ":" + obj.get(1) + ":" + obj.get(2));
            }
            break;

        case "qyrv_queryalgo": // 查询 algo
            DynamicObjectCollection collections4 =
                QueryServiceHelper.query(this.getClass().getName(), "qyrv_triptype",
                    "number,name,status", new QFilter[] { new QFilter("status",
                    QCP.equals, "A") }, "number desc");

            Iterator<DynamicObject> it4 = collections4.iterator();
            while (it4.hasNext()) {
                DynamicObject obj = it4.next();
                System.out.println(obj.get(0) + ":" + obj.get(1) + ":" + obj.get(2));
            }
            break;

```

```

        case "qyrv_querydataset": // 查询返回 DataSet
            DataSet ds = QueryServiceHelper.queryDataSet(this.getClass().getName(),
                "qyrv_triptype",
                "number,name,status", new QFilter[] { new QFilter("status",
                QCP.equals, "A") }, "number desc");

            Iterator<Row> it5 = ds.iterator();
            while (it5.hasNext()) {
                Row row = it5.next();
                System.out.println(row.get(0) + ":" + row.get(1) + ":" + row.get(2));
            }
            break;

```

```

        case "qyrv_queryone":// 查询单条数据
            DynamicObject do2 = QueryServiceHelper.queryOne("qyrv_triptype",
"number,name,status",
                new QFilter[] { new QFilter("status", QCP.equals, "A") });
            System.out.println(do2.get(0) + ":" + do2.get(1) + ":" + do2.get(2));
            break;
        case "qyrv_queryprimarykeys":// 查询主键 Keys
            List<Object> list =
QueryServiceHelper.queryPrimaryKeys("qyrv_triptype",
                new QFilter[] { new QFilter("status", QCP.equals, "A") }, "number
desc", 2);

            for (Object obj : list) {
                System.out.println(obj);
            }
            break;
    }
}

```

注册插件
×

插件路径*

kd.qyrv.orm.QueryServiceHelperDemo

描述说明*

测试QueryServiceHelper

是否启用

☒

取消

确定

注册插件

2-3.实体操作服务帮助类 OperationServiceHelper

kd.bos.servicehelper.operation.OperationServiceHelper 实体操作服务类为实体操作的

上层封装，简化实体操作服务调用操作。

举例，调用提交操作：

```
OperationResult result = OperationServiceHelper.executeOperate("submit", FormI  
d_Bill2, new DynamicObject[] { billObj2 }, OperateOption.create());
```

方法	说明
static OperationResult executeOperate(String operationKey, String entityNumber, DynamicObject[] dataEntities, OperateOption option)	执行操作
static OperationResult executeOperate(String operationKey, String entityNumber, Object[] ids, OperateOption option)	执行操作

```

public class OperationServiceHelperDemo extends AbstractListPlugin {

    @Override
    public void registerListener(EventObject e) {
        super.registerListener(e);
        this.addClickListeners(new String[] { "qyrv_unaudit" });
    }

    @Override
    public void itemClick(ItemClickEvent evt) {
        super.itemClick(evt);
        String key = evt.getItemKey();
        if ("qyrv_unaudit".equals(key)) {
            //查询编号为 CCLX0001 的数据
            DynamicObject[] objs = BusinessDataServiceHelper.load("qyrv_triptype",
"number,name,status,enable,createorg",
                new QFilter[] { new QFilter("number", QCP.equals, "CCLX0001") });
            //反审核
            OperationResult result = OperateServiceHelper.executeOperate("unaudit",
"qyrv_triptype", objs,
                OperateOption.create());
            String message = result.getMessage();
            System.out.println("message:" + message);
            this.getView().showMessage(message);
        }
    }
}

```

演示效果：

执行操作之前，编号为 CCLX0001 的数据状态为已审核。

插件代码执行之后，编号为 CCLX0001 的数据状态为暂存。

出差类型列表

出差类型

常用条件过滤

业务组织

环宇国际集团;

Q

使用状态

可用

新增

删除

提交

审核

禁用

更多

刷新

分配

管理

退出

已选1条, 共5条

☐	#	编码	名称	使用状态	备注	数据状态
☒	1	CCLX0001	出差类型测试	可用	通过操作服务添加的出差类型	已审核
☐	2	CCLX0002	Postman添加的出差类型	可用	Postman通过操作服务API添加的出差类型	暂存
☐	3	TRIPTYPE-0001	临时出差	可用	临时出差是员工临时到常住地以外地区的公务出差	已审核
☐	4	TRIPTYPE-0002	出长差	可用	出长差是出差到同一地点且行程超过10天(含10天)的出差	已审核
☐	5	TRIPTYPE-0003	驻外事务	可用	驻外事务是工程建设人员派驻到工程项目现场且出差行程超...	已审核

出差类型列表

出差类型

常用条件过滤

业务组织

环宇国际集团;

Q

使用状态

可用

新增

删除

提交

审核

禁用

更多

刷新

分配

管理

退出

共5条

☐	#	编码	名称	使用状态	备注	数据状态
☐	1	CCLX0001	出差类型测试	可用	通过操作服务添加的出差类型	暂存
☐	2	CCLX0002	Postman添加的出差类型	可用	Postman通过操作服务API添加的出差类型	暂存
☐	3	TRIPTYPE-0001	临时出差	可用	临时出差是员工临时到常住地以外地区的公务出差	已审核
☐	4	TRIPTYPE-0002	出长差	可用	出长差是出差到同一地点且行程超过10天(含10天)的出差	已审核
☐	5	TRIPTYPE-0003	驻外事务	可用	驻外事务是工程建设人员派驻到工程项目现场且出差行程超...	已审核

2-4.业务数据服务帮助类 BusinessDataServiceHelper

kd.bos.servicehelper.BusinessDataServiceHelper 业务数据服务帮助类提供查询、缓存查询等功能。

举例，生成实体主键：

```
DynamicObject obj = BusinessDataServiceHelper.loadSingleFromCache(pkId, mainEntityType);
```


限定符和类型	方法和说明
public static Object[]	load (Object[] pkArray, IDataEntityType type) 批量读取实体数据
public static DynamicObject[]	load (Object[] pkArray, DynamicObjectType type) 批量读取实体数据
public static DynamicObject[]	load (String entityName, String selectProperties, QFilter[] filters) 按条件读取实体数据
public static DynamicObject[]	load (String entityName, String selectProperties, QFilter[] filters, String orderBy) 按条件读取实体数据
public static DynamicObject[]	load (String entityName, String selectProperties, QFilter[] filters, String orderBy, int top) 按条件读取实体数据，取top张
public static DynamicObject[]	load (String entityName, String selectProperties, QFilter[] filters, String orderBy, int pageIndex, int pageSize) 按条件读取实体数据，在符合条件的数据中分页取数
public static java.util.Map<Object,DynamicObject>	loadFromCache (Object[] pkArray, DynamicObjectType type) 批量读取实体数据，使用缓存
public static java.util.Map<Object,DynamicObject>	loadFromCache (Object[] pkArray, String entityName) 批量读取实体数据，使用缓存
public static java.util.Map<Object,DynamicObject>	loadFromCache (String entityName, String selectProperties, QFilter[] filters) 按条件读取实体数据，使用缓存
public static java.util.Map<Object,DynamicObject>	loadFromCache (String entityName, String selectProperties, QFilter[] filters, String orderBy) 按条件读取实体数据，使用缓存
public static java.util.Map<Object,DynamicObject>	loadFromCache (String entityName, QFilter[] filters) 按条件读取实体数据，使用缓存
public static java.util.Map<Object,DynamicObject>	loadFromCache (DynamicObjectType type, QFilter[] filters) 按条件读取实体数据，使用缓存
public static DynamicObject	loadHead (Object pk, DynamicObjectType type) 读取实体数据，仅含根实体数据（单据头），不含分录数据
public static void	loadRefence (Object[] dataEntitys, IDataEntityType type) 加载实体数据中引用的基础数据，填充到实体数据包中
public static DynamicObject	loadSingle (Object pk, DynamicObjectType type) 读取单张实体数据
public static DynamicObject	loadSingle (Object pk, DynamicObjectType type, int start, int limit) 读取单张实体数据，指定分录起始行
public static DynamicObject	loadSingle (Object pk, String entityName) 读取单张实体数据
public static DynamicObject	loadSingle (Object pk, String entityName, String selectProperties) 读取单张实体数据，指定读取字段范围
public static DynamicObject	loadSingle (String entityName, String selectProperties, QFilter[] filters) 读取第一张符合条件的实体数据，指定读取字段范围
public static DynamicObject	loadSingleFromCache (Object pk, String entityName) 读取单张实体数据，使用缓存
public static DynamicObject	loadSingleFromCache (Object pk, String entityName, String selectProperties) 读取单张实体数据，使用缓存
public static DynamicObject	loadSingleFromCache (String entityName, QFilter[] filters) 读取第一张符合条件的实体数据，使用缓存
public static DynamicObject	loadSingleFromCache (String entityName, String selectProperties, QFilter[] filters) 读取第一张符合条件的实体数据，使用缓存
public static DynamicObject	loadSingleFromCache (Object pk, DynamicObjectType type) 读取单张实体数据，使用缓存
public static DynamicObject[]	loadWithPermission (String entityName, String selectProperties, QFilter[] filters, String orderBy, int top, String appld) 加载有权的数据
public static DynamicObject	newDynamicObject (String entityName) 通过实体标识创建一个动态实体数据包
public static void	removeCache (DynamicObjectType type) 清理缓存的实体数据

```

public class BusinessDataServiceHelperDemo extends AbstractFormPlugin {
    @Override
    public void beforeBindData(EventObject e) {
        super.beforeBindData(e);
        //1438203153899061248 是记录的 fid 值
        Map<Object, DynamicObject> map = BusinessDataServiceHelper.loadFromCache(new
Object[] { "1438203153899061248" },
            this.getModel().getDataEntity().getDynamicObjectType());
        Collection<DynamicObject> collections = map.values();
        Iterator<DynamicObject> it = collections.iterator();
        while(it.hasNext()) {
            DynamicObject obj = it.next();
            System.out.println(obj.get(0)+":"+obj.get(1)+":"+obj.get(2));
        }
        System.out.println(this.getModel().getDataEntity().getDynamicObjectType());
    }
}

```

2-5.实体保存服务帮助类 SaveServiceHelper

kd.bos.servicehelper > kd.bos.servicehelper.operation

类 SaveServiceHelper

@KSOBJECT

@SdkService(name="单据保存服务")

public class **SaveServiceHelper**

实体保存服务

限定符和类型	方法和说明
public static void	clearDataEntityCache (String entityNumber) 清理ORM实体数据缓存
public static Object[]	save (IDataEntityType type, Object[] dataEntities) 保存数据：直接保存入库，不走操作校验
public static Object[]	save (IDataEntityType type, Object[] dataEntities, OperateOption option) 保存数据：直接保存入库，不走操作校验
public static Object[]	save (IDataEntityType type, Object[] dataEntities, boolean commitSnapData) 保存数据：直接保存入库，不走操作校验
public static Object[]	save (DynamicObject[] dataEntities) 保存数据：直接保存入库，不走操作校验
public static Object[]	save (DynamicObject[] dataEntities, OperateOption option) 保存数据：直接保存入库，不走操作校验
public static Object[]	save (DynamicObject[] dataEntities, boolean commitSnapData) 保存数据：直接保存入库，不走操作校验
public static OperationResult	saveOperate (String entityNumber, DynamicObject[] dataEntities, OperateOption option) 调用单据保存操作：走操作校验及插件，通过后才保存入库
public static OperationResult	saveOperate (String operationKey, String entityNumber, DynamicObject[] dataEntities, OperateOption option) 调用单据保存操作：走操作校验及插件，通过后才保存入库
public static void	update (DynamicObject[] dataEntities, OperateOption option) 更新数据：直接更新入库，不走操作校验
public static void	update (DynamicObject dataEntity) 更新数据
public static void	update (DynamicObject dataEntity, OperateOption option) 更新数据

2-6.删除实体操作服务类 DeleteServiceHelper

[kd.bos.servicehelper](#) > [kd.bos.servicehelper.operation](#)

类 DeleteServiceHelper

@KSOject

@SdkService(name="单据删除服务")

```
public class DeleteServiceHelper
```

删除实体操作服务类

限定符和类型	方法和说明
public static void	<code>delete(IDataEntityType type, Object[] pks)</code> 根据实体类型和实体ID删除数据：直接删库，不走删除校验
public static int	<code>delete(String entityName, QFilter[] qFilters)</code> 根据实体名称和过滤条件删除数据：直接删库，不走删除校验
public OperationResult	<code>deleteOperate(String operationKey, String entityName, Object[] pks, OperateOption option)</code> 调用单据删除操作服务：执行删除校验及插件，校验通过后才删库
public OperationResult	<code>deleteOperate(String entityName, Object[] pks, OperateOption option)</code> 调用单据删除操作服务

三、操作实践

设计学员信息表，信息如下：

名称	字段名	类型
学号	sno	整形
姓名	sname	字符串
性别	sex	字符串
年龄	age	整形
地址	address	字符串
联系方式	telephone	字符串
学历	education	字符串
邮箱	email	字符串

实现如下需求：

- 1、手动添加学员信息
- 2、查询所有学员信息打印在控制台
- 3、根据学员编号删除学员信息
- 4、将所有学员的年龄增加 1 岁
- 5、修改姓名为 “张三”，学员的地址为 “湖南长沙”